Task 1 : Load the data and build the tree (insert)

(a) Read airports.dat line by line, split each line into fields (careful: names can contain commas inside quotes), skip lines whose IATA code is \N, and store the records in an array (or vector) of Airport.

Pseudocode:

```
        function splitLine(line)
1:          fields = []
2:          field = “”
3:          insideQuotes = false
4:          for i=0, i < line.length()
5:              if line[i] = ""
6:                  insideQuotes = !insideQuotes
7:              else if line[i] = ',' AND insideQuotes = false
8:                  fields.push_back(field)
9:                  field = ""
10:             else
11:                 field = field + line[i]
12:         fields.push_back(field)
13:         return fields
        end function

        function loadAirports(lines, airports)
1:          for i=0, i < lines.size()
2:              fields = splitLine(lines[i])
3:              if fields[4] = "\N"
4:                  continue
5:              Airport a
6:              a.name = fields[1]
7:              a.city = fields[2]
8:              a.country = fields[3]
9:              a.iata = fields[4]
10:             a.icao = fields[5]
11:             a.latitude = fields[6]
12:             a.longitude = fields[7]
13:             airports.push_back(a)
        end function
```

(b) Build the BST by inserting the records in file order. Count how many keys were inserted and report any duplicate (rule I4).

Pseudocode:

```
        function insert(AirportBST tree, Airport a)
1:          if tree.root = NULL
2:              tree.root = new Node(a)
3:              return true
4:          current = tree.root
5:          while true
6:              if a.iata < current.data.iata
7:                  if current.left = NULL
8:                      current.left = new Node(a)
9:                      return true
10:                 current = current.left
11:             else if a.iata > current.data.iata
12:                 if current.right = NULL
13:                     current.right = new Node(a)
14:                     return true
15:                 current = current.right
16:             else
17:                 return false
        end function


        function buildTree(airports, tree)
1:          inserted = 0
2:          duplicates = 0
3:          for i=0, i < airports.size()
4:              if insert(tree, airports[i]) = true
5:                  inserted++
6:              else
7:                  duplicates++
8:          return inserted, duplicates
        end function
```

(c) Report the height of the tree and the average depth of a node.

Pseudocode:

```
        function height(Node* node)
1:          if node = NULL
```

```
2:                  return -1
3:          leftHeight = height(node.left)
4:          rightHeight = height(node.right)
5:          if leftHeight > rightHeight
6:                  return leftHeight + 1
7:          else
8:                  return rightHeight + 1
    end function

    function calculateDepth(Node* node, depth, totalDepth, count)
1:          if node = NULL
2:                  return
3:          totalDepth = totalDepth + depth
4:          count++
5:          calculateDepth(node.left, depth + 1, totalDepth, count)
6:          calculateDepth(node.right, depth + 1, totalDepth, count)
    end function


    function averageDepth(root)
1:          totalDepth = 0
2:          count = 0
3:          calculateDepth(root, 0, totalDepth, count)
4:          if count = 0
5:                  return 0
6:          return totalDepth / count
    end function
```

Task 2 : Search

(a) find CODE prints the airport and the search path (every code compared) and the number of comparisons.

Pseudocode:

```
    function search(code)
1:          current = root
2:          comparisons = 0
3:          path = ""
4:          while current != NULL
5:                  comparisons++
```

```
6:                      path = path + current.data.iata
7:                      if code = current.data.iata
8:                              print path
9:                              print comparisons
10:                             print airport information
11:                             return true
12:                     else if code < current.data.iata
13:                             current = current.left
14:                     else
15:                             current = current.right
16:             print path
17:             print comparisons
18:             print “Airport not found”
19:             return false
        end function
```

(b) Test every search case: the root, a leaf, a code deep in the tree, a code that does not exist (for example XYZ), a code smaller than every key and one larger than every key. Also print the minimum and maximum code.

Pseudocode:

```
        function findMin()
1:              if root = NULL
2:                      return NULL
3:              current = root
4:              while current.left != NULL
5:                      current = current.left
6:              return current.data.iata
        end function

        function findMax()
1:              if root = NULL
2:                      return NULL
3:              current = root
4:              while current.right != NULL
5:                      current = current.right
6:              return current.data.iata
        end function

        function findLeaf(Node* node)
1:              if node = NULL
```

```
2:                  return false
3:          if node.left = NULL AND node.right = NULL
4:                  print node.data.iata
5:                   return true
6:          if findLeaf(node.left) = true
7:                  return true
8:          if findLeaf(node.right) = true
9:                  return true
10:         return false
    end function

    function findDeepest(Node* node, depth, maxDepth, code)
1:          if node = NULL
2:                  return
3:          if depth > maxDepth
4:                  maxDepth = depth
5:                  code = node.data.iata
6:          findDeepest(node.left, depth + 1, maxDepth, code)
7:          findDeepest(node.right, depth + 1, maxDepth, code)
    end function


    function getDeepestCode()
1:          maxDepth = -1
2:          code = ""
3:          findDeepest(root, 0, maxDepth, code)
4:          return code
    end function
```

Task 3 : Delete

(a) remove CODE deletes an airport (for example when it closes) using the three deletion cases.

Pseudocode:

```
    function findSuccessor(Node* node)
1:          current = node.right
2:          while current.left != NULL
3:                  current = current.left
4:          return current
```

```
        end function

        function remove(code)
1:              current = root
2:              parent = NULL
3:              while current != NULL AND current.data.iata != code
4:                      parent = current
5:                      if code < current.data.iata
6:                              current = current.left
7:                      else
8:                              current = current.right
9:              if current = NULL
10:                     return false
11:             if current.left = NULL AND current.right = NULL
12:                     if current = root
13:                             root = NULL
14:                     else if parent.left = current
15:                             parent.left = NULL
16:                     else
17:                             parent.right = NULL
18:                     delete current
19:             else if current.left = NULL OR current.right = NULL
20:                     if current.left != NULL
21:                             child = current.left
22:                     else
23:                             child = current.right
24:                     if current = root
25:                             root = child
26:                     else if parent.left = current
27:                             parent.left = child
28:                     else
29:                             parent.right = child
30:                     delete current
31:             else
32:                     successor = findSuccessor(current)
33:                     successorParent = current
34:                     while successorParent.left != successor
35:                             successorParent = successorParent.left
36:                     current.data = successor.data
37:                     if successor.right != NULL
```

```
38:                             successorParent.left = successor.right
39:                     else
40:                             successorParent.left = NULL
41:                     delete successor
42:             return true
        end function
```

(b) Find, in your tree, one code for each case and test it: a leaf (D1), a node with only a left child, a node with only a right child (D2), a node with two children (D3) and the root. Print the level order of the top three levels before and after each delete.

Pseudocode:

```
        function printLevelOrder()
1:              if root = NULL
2:                      print "Tree is empty"
3:                      return
4:              queue = [root]
5:              level = 0
6:              while queue is not empty AND level < 3
7:                      levelSize = queue.size() - index
8:                      for i=0, i < levelSize
9:                              current = queue[index]
10:                             print current.data.iata
11:                             if current.left != NULL
12:                                     queue.push_back(current.left)
13:                             if current.right != NULL
14:                                     queue.push_back(current.right)
15:                             index++
16:                     level++
        end function

        function testDeletion(code)
1:              print "Before deletion"
2:              printLevelOrder()
3:              remove(code)
4:              print "After deletion"
5:              printLevelOrder()
        end function
```

(c) add a made-up airport (code QQQ, your own name and city) and show it can be found, then removed.

Task 4 : Traversals and validation

(a) list prints every airport in alphabetical code order (inorder). Check that it is sorted.

Pseudocode:

```
    function inorder(Node* node)
1:      if node = NULL
2:          return
3:      inorder(node.left)
4:      print node.data.iata
5:      inorder(node.right)
    end function

    function checkSorted(codes)
1:      for i=1, i < codes.size()
2:          if codes[i] < codes[i-1]
3:              return false
4:      return true
    end function
```

(b) save FILE writes the codes in preorder; load FILE rebuilds the tree from that file. Show that the reloaded tree has the same shape (same height and level order).

Pseudocode:

```
    function preorder(Node* node, file)
1:      if node = NULL
2:          return
3:      write node.data to file
4:      preorder(node.left, file)
5:      preorder(node.right, file)
    end function

    function saveTree(filename)
1:      file = open(filename, write)
2:      if file = NULL
3:          return false
4:      preorder(root, file)
```

```
5:          close(file)
6:          return true
        end function

        function loadTree(filename)
1:          file = open(filename, read)
2:          if file = NULL
3:              return false
4:          clearTree(root)
5:          while file has data
6:              read airport record
7:              insert(airport)
8:          close(file)
9:          return true
        end function

        function compareTrees(Node* first, Node* second)
1:          if first = NULL AND second = NULL
2:              return true
3:          if first = NULL OR second = NULL
4:              return false
5:          if first.data.iata != second.data.iata
6:              return false
7:          leftSame = compareTrees(first.left, second.left)
8:          rightSame = compareTrees(first.right, second.right)
9:          return leftSame AND rightSame
        end function
```

(c) validate checks the BST property with the range (lo, hi) method after every insert and delete in your tests.

Pseudocode:

```
        function validate(Node* node, string lo, string hi)
1:          if node = NULL
2:              return true
3:          if node.data.iata <= lo OR node.data.iata >= hi
4:              return false
5:          if validate(node.left, lo, node.data.iata) = false
6:              return false
7:          if validate(node.right, node.data.iata, hi) = false
```

```
8:                      return false
9:              return true
        end function

        function validateTree()
1:              return validate(root, "", "ZZZ")
        end function
```

Task 5 : Range queries

range FROM TO lists every airport with a code from FROM to TO, without visiting subtrees that are outside the range. Report how many nodes you visited. For example range BOS BWI must return 113 airports.

Pseudocode:

```
        function rangeSearch(Node* node, string from, string to, int& visited)
1:              if node = NULL
2:                      return
3:              visited++
4:              if node.data.iata > from
5:                      rangeSearch(node.left, from, to, visited)
6:              if node.data.iata >= from AND node.data.iata <= to
7:                      print node.data.iata
8:              if node.data.iata < to
9:                      rangeSearch(node.right, from, to, visited)
        end function

        function range(from, to)
1:              if from > to
2:                      print "Invalid range"
3:                      return
4:              visited = 0
5:              rangeSearch(root, from, to, visited)
6:              print visited
        end function
```

Task 6 : The shape experiment

Build the tree in three ways from the same 6,072 codes and fill in the table: (1) file order, (2) sorted order, (3) balanced build from the sorted array (middle element as the root, a rebalance command). For each, also time 10,000 random searches.

Pseudocode:

```
        function buildSortedTree(airports)
1:          sortedAirports = copy(airports)
2:          sort(sortedAirports by iata)
3:          tree = new AirportBST()
4:          for i = 0 to sortedAirports.size() - 1
5:              insert(tree, sortedAirports[i])
6:          return tree
        end function

        function buildBalanced(sortedAirports, start, end)
1:          if start > end
2:              return NULL
3:          mid = (start + end) / 2
4:          node = new Node(sortedAirports[mid])
5:          node.left = buildBalanced(sortedAirports, start, mid - 1)
6:          node.right = buildBalanced(sortedAirports, mid + 1, end)
7:          return node
        end function

        function buildRandomTree(airports)
1:          shuffledAirports = copy(airports)
2:          shuffle(shuffledAirports)
3:          tree = new AirportBST()
4:          for i = 0 to shuffledAirports.size() - 1
5:              insert(tree, shuffledAirports[i])
6:          return tree
        end function

        function timeSearches(tree, airports)
1:          total = 10000
2:          indexes = generateRandomIndexes(airports.size(), total)
3:          startTime = currentTime()
4:          for i = 0 to total - 1
5:              code = airports[indexes[i]].iata
```

```
6:                  search(tree, code)
7:          endTime = currentTime()
8:          elapsedTime = endTime - startTime
9:          return elapsedTime
    end function
```

Build Order	**Height**	**Average depth**	**Time for 10,000 searches**	**Your explanation**
File order	177	18.3233	34.6661ms	The original file order produces a somewhat unbalanced tree because the airport records are not randomly ordered.
Sorted order	6071	3035.5	2633.41 ms	Inserting sorted codes creates a chain-like tree because each new code is greater than the previous one.
Balanced (middle rule)	12	10.6532	25.8926 ms	Choosing the middle element as the root creates a balanced tree with shorter search paths.
Random shuffle (bonus)	28	13.7269	31.5726 ms	Randomizing the insertion order creates a relatively balanced tree, reducing its height and improving search performance.

Question: Explain why the file order is so much worse than a random shuffle (about 25 to 29). Hint: look at how the lines of the file are ordered.

Answer: The file order is much worse than a random shuffle because the airport database is not arranged in a completely random order. When these airports are inserted into a BST in file order, certain patterns in the insertion sequence cause the tree to become unbalanced. This creates longer branches and increases the number of comparisons required for some searches.

Randomly shuffling the airport records changes the insertion order and reduces these patterns. So, the tree becomes more balanced, with a height of 28 compared to 177 for the original file order.